



Quick Start - Dominion Arena Tática



Status da Aplicação

A aplicação está **100% pronta** para ser executada! Todas as verificações passaram com sucesso.



Requisitos do Sistema

Antes de começar, certifique-se que você tem instalado:

- **Node.js** 16+ ([baixar aqui](https://nodejs.org/) (https://nodejs.org/))
- **Docker** e **Docker Compose** ([baixar aqui](https://docker.com/) (https://docker.com/))
- **Git** (para clonar o repositório)

Verificar Instalações

```
node --version      # Deve ser v16 ou superior
npm --version       # Deve estar instalado
docker --version    # Deve estar instalado
docker compose version # Deve estar instalado (Docker Compose V2)
```



Como Rodar o Jogo

Opção 1: Usando o Script de Inicialização (Recomendado)

A forma mais fácil é usar o script `start.sh` que automatiza tudo:

```
# 1. Abra o terminal na pasta do projeto
cd dominion_arena_tatica

# 2. Tornar o script executável
chmod +x start.sh

# 3. Rodar o script
./start.sh
```

O script vai:

- ☒ Iniciar PostgreSQL em Docker
- ☒ Instalar dependências do backend
- ☒ Iniciar o servidor backend (porta 5000)
- ☒ Instalar dependências do frontend
- ☒ Iniciar o servidor frontend (porta 3000)

Aguarde a mensagem:

```
=====
Dominion Arena Tatica iniciado com sucesso!
=====

Backend: http://localhost:5000
Frontend: http://localhost:3000
```

Abra seu navegador e acesse: <http://localhost:3000>

Opção 2: Instalação Manual (Controle Total)

Se preferir mais controle, siga os passos abaixo em três terminais diferentes:

Terminal 1: PostgreSQL com Docker Compose

```
cd dominion_arena_tatica
docker compose up -d postgres
sleep 5 # Aguarde o banco iniciar
```

Terminal 2: Backend

```
cd dominion_arena_tatica/backend
npm install
npm run dev
```

Você verá:

```
Database initialized
Server running on port 5000
```

Terminal 3: Frontend

```
cd dominion_arena_tatica/frontend
npm install
npm run dev
```

Você verá:

```
✓ Ready in 2.1s
→ Local: http://localhost:3000
```

Usando o Jogo

Primeira Execução

1. Abra <http://localhost:3000> no navegador

2. Crie um novo jogo:

- Digite seu nome de usuário (ex: "Jogador1")
- Clique em "Criar Novo Jogo"
- Você receberá um ID do jogo (ex: a1b2c3d4-e5f6-...)

3. Convide amigos (opcional):

- Compartilhe o ID da loja
- Seus amigos acessam <http://localhost:3000>
- Digitam o mesmo ID do jogo
- Digitam seus nomes

4. Comece a jogar:

- Clique "Rolar Dados" para começar
- Siga as fases do jogo: Roll → Buy → Combat → End
- Compre cartas, coloque-as no campo e batalhe!



Verificar se Está Funcionando

Backend

```
curl http://localhost:5000/api/health
# Resposta esperada: {"status":"ok"}
```

Frontend

Abra <http://localhost:3000> no navegador. Deve aparecer a página de entrada.

Banco de Dados

```
# Conectar ao banco (se tiver psql instalado)
psql -U postgres -h localhost -d dominion_arena

# Comandos úteis:
# \dt - Listar tabelas
# \q - Sair
```



Solução de Problemas

Porta 3000 já está em uso

```
# Linux/Mac
lsof -i :3000
kill -9 <PID>

# Windows
netstat -ano | findstr :3000
taskkill /PID <PID> /F
```

Porta 5000 já está em uso

```
# Linux/Mac
lsof -i :5000
kill -9 <PID>

# Windows
netstat -ano | findstr :5000
taskkill /PID <PID> /F
```

Docker Compose não encontrado

```
# Tente atualizar Docker
docker --version # Deve ser Docker 20.10+ com Compose V2

# Ou instale Docker Compose V2
curl -L "https://github.com/docker/compose/releases/download/v2.20.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose
```

WebSocket não conecta

1. Verifique se o backend está rodando em `http://localhost:5000`
2. Verifique se não há firewall bloqueando a porta 5000
3. Verifique `.env.local` :
`NEXT_PUBLIC_API_URL=http://localhost:5000`
`NEXT_PUBLIC_SOCKET_URL=http://localhost:5000`
4. Reinicie o frontend com `npm run dev`

Banco de dados não conecta

```
# Verificar se PostgreSQL está rodando
docker ps | grep postgres

# Se não estiver:
cd dominion_arena_tatica
docker compose up -d postgres

# Aguarde 5-10 segundos e tente novamente
```

Arquivos Importantes

- **Backend:** `/backend/server.js` - Servidor principal
 - **Frontend:** `/frontend/src/pages/index.js` - Página inicial
 - **Jogo:** `/frontend/src/pages/game/[gameId].js` - Página do jogo
 - **Cartas:** `/dominion_cards.json` - Dados das 40 cartas
 - **Banco:** `/docker-compose.yml` - Configuração PostgreSQL
 - **Config:** `/backend/.env` e `/frontend/.env.local` - Variáveis
-

Documentação Completa

- **SETUP.md** - Guia detalhado de instalação
 - **MECANICAS.md** - Regras e mecânicas do jogo
 - **DESENVOLVIMENTO.md** - Guia para adicionar features
 - **FEATURES.md** - Lista completa de funcionalidades
 - **README.md** - Visão geral do projeto
-

Logs e Debug

Ver logs do Backend

```
# Se rodando com npm run dev, você verá:  
# - Conexões recebidas  
# - Erros de banco de dados  
# - Eventos de Socket.io
```

Ver logs do Frontend

```
# Abra o Console do Navegador (F12)  
# Você verá:  
# - Erros de conexão WebSocket  
# - Eventos de jogo  
# - Dados recebidos
```

Ativar modo desenvolvimento

```
# No arquivo /backend/.env:  
NODE_ENV=development # Já está assim por padrão  
  
# Isso ativa:  
# - Logs detalhados  
# - Nodemon para reload automático  
# - CORS habilitado
```

Atalhos Úteis

Ação	Como Fazer
Parar tudo	Pressione <code>Ctrl+C</code> em todos os terminais
Reiniciar Backend	<code>Ctrl+C</code> no terminal do backend, depois <code>npm run dev</code>
Limpar cache Frontend	<code>npm run build</code> para reconstruir
Ver erros de Build	<code>npm run build 2>&1 grep error</code>
Resetar Banco	<code>docker compose down -v</code> depois <code>docker compose up -d postgres</code>

Suporte

Se encontrar problemas:

1.  Verifique se Node.js, Docker e Git estão instalados
2.  Verifique se as portas 3000, 5000 e 5432 estão disponíveis
3.  Verifique os arquivos `.env` e `.env.local`
4.  Verifique os logs no console/terminal
5.  Leia os guias de documentação fornecidos

Pronto!

Agora você tem a aplicação rodando localmente. Divirta-se jogando Dominion Arena Tática! 

Para mais informações sobre o jogo, leia `MECANICAS.md`.

Para desenvolvemos novas features, leia `DESENVOLVIMENTO.md`.